

useContext



Storage



Biometría

Gestión de Sesión

en React Native

Persistencia · Seguridad · Experiencia de Usuario

Agenda

01



useContext

Estado global · AuthContext · Consumo en componentes

02



Storage

AsyncStorage · SecureStore · Integración con Context

03



Biometría

expo-local-authentication · Disponibilidad · Prompt

¿Por qué necesitamos estado global?

❌ El problema: Prop Drilling

```
App (tiene el user)
  ↓ pasa user como prop
  Navigator
    ↓ pasa user como prop
    HomeScreen
      ↓ pasa user como prop
      Header  ← lo necesita acá
```

Cada nivel intermedio recibe y pasa props que no usa → código frágil, difícil de mantener.

✅ La solución: Context API

```
AuthContext.Provider
  ├─ Navigator
  │    ├─ HomeScreen
  │    │    └─ Header ← accede directo
  │    └─ ProfileScreen ← accede directo
  └─ SettingsScreen ← accede directo
```

Cualquier componente consume el contexto directamente, sin pasar props intermedias.

¿Qué es useContext? — y las alternativas

React Context API es la solución nativa para compartir estado sin prop drilling. Según la complejidad de tu app, existen otras opciones:

Herramienta	Curva de aprendizaje	Boilerplate	Ideal para
useContext	Baja	Mínimo	Auth, tema, idioma — estados simples
Zustand	Baja-Media	Bajo	Apps medianas, muy popular en RN
Redux Toolkit	Media-Alta	Moderado	Apps grandes, estado muy complejo
Jotai / Recoil	Media	Bajo	Estado atómico y granular

💡 ***Para este curso usamos useContext — es nativo, sin dependencias y suficiente para manejar la sesión.***

Creando el AuthContext

```
// context/AuthContext.js
import { createContext, useState, useContext } from 'react';

export const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);

  const login = (userData) => setUser(userData);
  const logout = () => setUser(null);

  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}
```

AuthContext.js

```
// App.js — envolver la app
import { AuthProvider } from './context/AuthContext';
import { NavigationContainer } from
'@react-navigation/native';

export default function App() {
  return (
    <AuthProvider>
      <NavigationContainer>
        {/* ... stacks / tabs ... */}
      </NavigationContainer>
    </AuthProvider>
  );
}
```

App.js

Consumiendo el contexto

```
// screens/HomeScreen.js
import { useContext } from 'react';
import { AuthContext } from '../context/AuthContext';

export function HomeScreen() {
  const { user, logout } = useContext(AuthContext);

  return (
    <View>
      <Text>Bienvenido, {user?.nombre}</Text>
      <Button title="Cerrar sesión" onPress={logout} />
    </View>
  );
}
```

🔍 Acceso directo

No importa qué tan profundo esté el componente en el árbol

🔄 Reactivo

Cuando cambia el contexto, todos los consumidores se re-renderizan automáticamente

🚪 Control de rutas

Si user === null → redirigir al Login; si user existe → mostrar Home

¿Qué pasa si se cierra la app?



El contexto vive solo en memoria RAM

Al cerrar o matar la app, todo el estado se pierde

Sin persistencia

1. Usuario hace login
2. Cierra la app
3. Reabre → debe loguearse de nuevo
4. Mala experiencia de usuario ❌

Con persistencia (Storage)

1. Login → token guardado en disco
2. Cierra la app
3. Reabre → token leído → auto-login
4. Experiencia fluida ✅

Storage en React Native

¿Cómo elegir la herramienta correcta?

	AsyncStorage	SecureStore
Encriptado	❌ No	✅ Sí (Keychain / Keystore)
Tamaño máximo	Sin límite definido	2 KB por valor
Ideal para	Preferencias, caché	Tokens JWT, credenciales
Instalación	@react-native-async-storage/async-storage	expo-secure-store
Síncrono	❌ Async	❌ Async

🔐 ***Regla de oro: si es sensible (tokens, contraseñas, datos de sesión) → siempre SecureStore.***

AsyncStorage — Uso básico

```
import AsyncStorage from '@react-native-async-storage/async-storage';

// Guardar
await AsyncStorage.setItem('theme', 'dark');
await AsyncStorage.setItem('user', JSON.stringify({ id: 1, nombre: 'Ana'
}));

// Leer
const theme = await AsyncStorage.getItem('theme');
const raw   = await AsyncStorage.getItem('user');
const user  = raw ? JSON.parse(raw) : null;

// Eliminar
await AsyncStorage.removeItem('theme');

// Limpiar todo
await AsyncStorage.clear();
```

⚠️ Siempre async/await

Todas las operaciones son asíncronas, nunca bloqueantes.

📦 Solo strings

Para objetos usar JSON.stringify() al guardar y JSON.parse() al leer.

🚀 Instalación

npx expo install @react-native-async-storage/async-storage

SecureStore — Uso básico

```
import * as SecureStore from 'expo-secure-store';

// Guardar (encriptado en Keychain / Keystore)
await SecureStore.setItemAsync('token', 'eyJhbGciOiJIUzI1...');
await SecureStore.setItemAsync('user', JSON.stringify(userData));

// Leer
const token = await SecureStore.getItemAsync('token');
const raw   = await SecureStore.getItemAsync('user');
const user  = raw ? JSON.parse(raw) : null;

// Eliminar
await SecureStore.deleteItemAsync('token');

// Instalación
// npx expo install expo-secure-store
```

¿Cómo funciona?

Android

Android Keystore System
(cifrado a nivel hardware)

iOS

Keychain Services
(cifrado con Face ID / Touch ID)

Límite: 2 KB por clave.
Para datos grandes, guardá el token aquí y el resto en AsyncStorage.

Integrando Storage + Context

```
// context/AuthContext.js — con persistencia
import { createContext, useState, useEffect } from 'react';
import * as SecureStore from 'expo-secure-store';

export const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  // Al montar: leer token guardado
  useEffect(() => {
    const restore = async () => {
      const token = await SecureStore.getItemAsync('token');
      if (token) setUser({ token });  // o decodificá el JWT
      setLoading(false);
    };
    restore();
  }, []);

  const login = async (token, userData) => {
    await SecureStore.setItemAsync('token', token);
    setUser(userData);
  };

  const logout = async () => {
    await SecureStore.deleteItemAsync('token');
    setUser(null);
  };
```

¿Y si necesito más? — WatermelonDB

AsyncStorage y SecureStore son clave-valor. ¿Qué pasa cuando necesitás datos relacionales, queries y reactividad?

Concepto	Room (Android Nativo)	WatermelonDB (React Native)
Definir estructura	`@Entity`	`tableSchema()`
Modelo de datos	`@Dao + clase Java/Kotlin`	`class extends Model`
Reactividad	`LiveData / Flow`	`observe()`
Motor	SQLite (abstraído)	SQLite (abstraído)
Lenguaje	Java / Kotlin	**JavaScript puro ✅**
Migración de schema	Migration classes	`migrations array`

💡 *WatermelonDB es lazy — solo carga en memoria los registros que necesitás, lo que lo hace muy performante en mobile.*

WatermelonDB — Paso a paso (JS puro)

1 Instalación

```
npx expo install @nozbe/watermelondb  @nozbe/with-observables  expo-sqlite
```

2 Schema (estructura de la tabla)

```
import { tableSchema, appSchema }
  from '@nozbe/watermelondb';

export const schema = appSchema({
  version: 1,
  tables: [ tableSchema({
    name: 'posts',
    columns: [
      { name: 'title',   type: 'string' },
      { name: 'body',    type: 'string' },
      { name: 'user_id', type: 'string' },
    ]
  })]
});
```

3 Modelo (clase JS, sin decoradores)

```
import { Model } from '@nozbe/watermelondb';

export class Post extends Model {
  static table = 'posts';
  // Los campos se leen desde el schema
  // No necesitás declararlos en la clase
  // Accedés con: post.title,
  //   post.body, post.user_id
}
```

4 Leer y escribir

```
// Leer todos los posts
const posts = await database.get('posts').query().fetch();

// Filtrar por campo
const mine = await database.get('posts').query(Q.where('user_id', userId)).fetch();

// Crear (siempre dentro de database.write)
await database.write(async () => {
  await database.get('posts').create(post => {
    post.title   = 'Hola mundo';
    post.body    = 'Contenido del post';
    post.user_id = '123';
  });
});
```

WatermelonDB — Schema, Migraciones y Database

Schema multi-tabla + Migraciones

```
// schema.js
export const schema = appSchema({
  version: 3,   // sube con cada cambio
  tables: [
    tableSchema({ name: 'users', columns: [
      { name: 'name',  type: 'string' },
      { name: 'email', type: 'string' },
    ]}),
    tableSchema({ name: 'posts', columns: [
      { name: 'title',   type: 'string' },
      { name: 'user_id', type: 'string' },
      { name: 'likes',   type: 'number' }, // v2
    ]}),
  ]
});

// migrations.js
export const migrations = schemaMigrations({
  migrations: [
    {
      toVersion: 2,
      steps: [ addColumns({
        table: 'posts',
        columns: [{ name: 'likes', type: 'number' }]
      })]
    },
    {
      toVersion: 3,
      steps: [ createTable({
        name: 'comments',
        columns: [{ name: 'body', type: 'string' }]
      })]
    },
  ]
});
```

Database — singleton (se crea una sola vez)

```
// database.js
import { Database } from '@nozbe/watermelondb';
import SQLiteAdapter from
  '@nozbe/watermelondb/adapters/sqlite';
import { schema } from './schema';
import { migrations } from './migrations';
import { Post } from './models/Post';
import { User } from './models/User';

const adapter = new SQLiteAdapter({
  schema, migrations
});

export const database = new Database({
  adapter,
  modelClasses: [Post, User],
});
```

```
// Toda escritura va dentro de write()
// Si algo falla → rollback automático
// Agrupa ops en 1 sola transacción SQLite

await database.write(async () => {
  const user = await database
    .get('users').create(u => {
      u.name  = 'Ana';
      u.email = 'ana@mail.com';
    });
  await database
    .get('posts').create(p => {
      p.title   = 'Mi primer post';
      p.user_id = user.id;
    });
  // Ambas ops: 1 sola transacción ✅
});
```

Biometría en React Native



expo-local-authentication

👆	😊	👁️
Huella digital	**Reconocimiento facial**	**Escaneo de iris**
`FINGERPRINT`	`FACIAL_RECOGNITION`	`IRIS`

```
npx expo install expo-local-authentication
```

Verificando disponibilidad biométrica

```
import * as LocalAuthentication from 'expo-local-authentication';

async function checkBiometrics() {
  // 1. ¿Tiene hardware biométrico?
  const hasHardware = await LocalAuthentication.hasHardwareAsync();

  // 2. ¿El usuario tiene biometría enrolada?
  const isEnrolled = await LocalAuthentication.isEnrolledAsync();

  // 3. ¿Qué tipos soporta?
  const types = await
LocalAuthentication.supportedAuthenticationTypesAsync();
  // types puede ser: [1] Fingerprint, [2] FaceID, [3] Iris

  return { hasHardware, isEnrolled, types };
}
```

⚠️ *Siempre verificar los tres antes de lanzar el prompt para evitar errores en runtime.*

`hasHardwareAsync()`

Verifica si el dispositivo tiene sensor biométrico físico.

`isEnrolledAsync()`

Verifica si el usuario registró al menos una credencial biométrica.

`supportedAuthentication TypesAsync()`

Retorna un array con los tipos disponibles (fingerprint, face, iris).

Lanzando el prompt biométrico

```
async function handleBiometricLogin() {
  const { hasHardware, isEnrolled } = await checkBiometrics();

  if (!hasHardware || !isEnrolled) {
    Alert.alert('No disponible', 'Biometría no configurada en el
dispositivo');
    return;
  }

  const result = await LocalAuthentication.authenticateAsync({
    promptMessage:  'Autenticá tu identidad',
    fallbackLabel:  'Usar contraseña',
    cancelLabel:    'Cancelar',
    disableDeviceFallback: false,
  });

  if (result.success) {
    // Autenticación exitosa — cargar sesión desde SecureStore
    const token = await SecureStore.getItemAsync('token');
    login(token);
  } else {
    // result.error: 'user_cancel' | 'lockout' | 'not_enrolled' ...
    Alert.alert('Error', result.error);
  }
}
```

Posibles resultados

`success: true`

Autenticación exitosa

`user_cancel`

Usuario canceló el prompt

`lockout`

Demasiados intentos fallidos

`not_enrolled`

Sin credenciales registradas

`system_cancel`

El SO interrumpió la auth

Errores comunes — Qué NO hacer

❌ AsyncStorage para tokens

Guardar JWT o passwords en AsyncStorage porque no está encriptado. Cualquier app con acceso root puede leerlo.

✅ **Siempre SecureStore para datos sensibles.**

❌ Un solo contexto global para todo

Meter user + theme + cart + config en un AppContext. Cada cambio re-renderiza TODA la app.

✅ **Separar en contextos específicos: AuthContext, ThemeContext, etc.**

❌ No verificar expiración del token

Restaurar el token del storage y usarlo sin verificar si ya expiró → llamadas 401 en runtime.

✅ **Decodificar el JWT y validar exp antes de setear el user en el contexto.**

❌ Lanzar biometría sin verificar disponibilidad

Llamar authenticateAsync() directamente sin hasHardware/isEnrolled → crash en dispositivos sin sensor.

✅ **Siempre verificar los tres checks antes del prompt.**

Manejo de token expirado — Interceptor Axios

```
// services/api.js
import axios from 'axios';

export const api = axios.create({ baseURL: 'https://mi-api.com' });

// REQUEST: adjunta el token en cada llamada
api.interceptors.request.use(async (config) => {
  const token = await SecureStore.getItemAsync('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// RESPONSE: detecta 401 y limpia la sesion
api.interceptors.response.use(
  (response) => response,
  async (error) => {
    const { logout } = useContext(AuthContext);
    if (error.response?.status === 401) {
      logout();
    }
    return Promise.reject(error);
  }
);
```

Flujo del interceptor



¿Qué problema tenemos?

El interceptor vive fuera del árbol de React.

¿Cómo llamamos a logout() del AuthContext desde services/api.js?

❌ **No podés usar useContext(AuthContext) acá — los hooks solo funcionan dentro de componentes React o custom hooks.**

Solución — Inyectar logout como parámetro en setupInterceptors()

```
// services/api.js — recibe logoutFn como parámetro
export function setupInterceptors(logoutFn) {
  api.interceptors.response.use(
    (response) => response,
    async (error) => {
      if (error.response?.status === 401) {
        await SecureStore.deleteItemAsync('token');
        logoutFn();   // <-- acá usamos el parámetro
      }
      return Promise.reject(error);
    }
  );
}

// context/AuthContext.js — registrar al montar el Provider
useEffect(() => { setupInterceptors(logout); }, []);
```

Best Practice — Separación de contextos

❌ Un solo contexto para todo

```
const AppContext = createContext();

// user + theme + cart + config + ...
// Cualquier cambio en theme
// re-renderiza TODA la app,
// incluso pantallas que solo
// necesitan el user
```

Un cambio en el carrito recarga
las pantallas de perfil y ajustes.
Rendimiento degradado con la app.

✅ Contextos separados

```
<AuthProvider>      // user, login, logout
 <ThemeProvider>    // darkMode, colors
  <CartProvider>    // items, addItem...
   <App />
  </CartProvider>
 </ThemeProvider>
</AuthProvider>

// Cada componente suscribe solo
// al contexto que necesita
```

Re-render quirúrgico: solo los
componentes suscritos al contexto
que cambió se actualizan.

Custom Hook — useAuth()

```
// hooks/useAuth.js
import { useContext } from 'react';
import { AuthContext } from '../context/AuthContext';

export function useAuth() {
  const context = useContext(AuthContext);

  if (!context) {
    throw new Error(
      'useAuth debe usarse dentro de <AuthProvider>'
    );
  }

  return context;
}
```

```
// ❌ Sin custom hook (verboso)
import { useContext }   from 'react';
import { AuthContext }  from '../context/AuthContext';

const { user, logout } = useContext(AuthContext);


// ✅ Con custom hook (limpio)
import { useAuth } from '../hooks/useAuth';

const { user, logout } = useAuth();


// También podés derivar lógica:
// const { isAuthenticated } = useAuth();
// Donde isAuthenticated = !!user
```

Ventajas del custom hook:

Encapsula el acceso al contexto · Valida uso dentro del Provider · Permite derivar lógica (isAuthenticated...) · Un solo import por componente